Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment 5

Name : Om Rajendra Bharambe
PRN : 124B1F001
Batch : A1
Date : 24/03/2026

Aim:

Write **cursor** code using control structures and exception handling.

Objectives:

- **To implement row-level processing** by using **Cursors** to iterate through result sets for generating detailed reports, such as flight manifests and citizen complaint summaries.
- **To apply procedural control structures** including LOOP, LEAVE, and IF statements to manage complex data workflows and conditional archiving of senior passengers and citizens.
- **To develop robust error handling mechanisms** using DECLARE CONTINUE HANDLER for SQLEXCEPTION and specific error codes (like 1062 for duplicates) to ensure system stability during data operations.
- **To ensure data integrity during administrative tasks** by creating safe deletion procedures that catch foreign key violations and log errors rather than allowing the application to crash.

Case Study: Airline Management System

SQL Queries:

1. Audit and Reporting Table Setup

These tables are designed to store logs for price changes and passenger manifest reports generated via cursors.

```
mysql> CREATE TABLE flight_audit_log(
    -> log_id INT AUTO_INCREMENT PRIMARY KEY,
    -> flight_id INT,
    -> old_price DECIMAL(10,2),
    -> new_price DECIMAL(10,2),
    -> change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE booking_report(
    -> report_id INT AUTO_INCREMENT PRIMARY KEY,
    -> flight_id INT,
    -> passenger_details VARCHAR(255));
Query OK, 0 rows affected (0.02 sec)
```

---

2. Procedure: Generate Flight Manifest (Using Cursors)

This procedure uses a cursor to iterate through passengers booked on a specific flight and populates the booking_report table.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GenerateFlightManifest(IN p_flight_id INT)
    -> BEGIN
    -> DECLARE done INT DEFAULT 0;
    -> DECLARE v_pname VARCHAR(50);
    -> DECLARE v_seat VARCHAR(5);
    -> DECLARE v_details VARCHAR(255);
    -> DECLARE manifest_cursor CURSOR FOR
    ->     SELECT p.pname, b.seat_no FROM passenger p
    ->     JOIN booking b ON p.passenger_id = b.passenger_id
    ->     WHERE b.flight_id = p_flight_id;
    -> DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    -> DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    -> BEGIN
    ->     INSERT INTO booking_report(flight_id, passenger_details) VALUES(p_flight_id,
'ERROR GENERATING MANIFEST');
    -> END;
```

```
    -> OPEN manifest_cursor;
    -> manifest_loop: LOOP
    ->     FETCH manifest_cursor INTO v_pname, v_seat;
    ->     IF done = 1 THEN
    ->         LEAVE manifest_loop;
    ->     END IF;
    ->     SET v_details = CONCAT('Passenger: ', v_pname, ', Seat: ', v_seat);
    ->     INSERT INTO booking_report(flight_id, passenger_details) VALUES(p_flight_id,
v_details);
    -> END LOOP;
    -> CLOSE manifest_cursor;
    -> END //
Query OK, 0 rows affected (0.03 sec)
```

---

3. Procedure: Passenger Archiving and Safe Deletion

The following procedures demonstrate **Cursor iteration** for archiving senior passengers and **Exception Handling** to catch foreign key constraint errors during deletion.

```
mysql> CREATE TABLE passenger_archive(
    -> archive_id INT AUTO_INCREMENT PRIMARY KEY,
    -> passenger_id INT UNIQUE,
    -> pname VARCHAR(50),
    -> age INT,
    -> archive_date DATE);
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE PROCEDURE ArchiveSeniorPassengers()
    -> BEGIN
    -> DECLARE done INT DEFAULT 0;
    -> DECLARE v_pid INT;
    -> DECLARE v_name VARCHAR(50);
    -> DECLARE v_age INT;
    -> DECLARE pass_cursor CURSOR FOR SELECT passenger_id, pname, age FROM passenger WHERE
age >= 30;
    -> DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    -> DECLARE CONTINUE HANDLER FOR 1062
    -> BEGIN
    ->     SET @duplicate_error = 1;
    -> END;
    -> OPEN pass_cursor;
    -> archive_loop: LOOP
    ->     FETCH pass_cursor INTO v_pid, v_name, v_age;
    ->     IF done = 1 THEN
    ->         LEAVE archive_loop;
    ->     END IF;
```

```
    ->     SET @duplicate_error = 0;
    ->     INSERT INTO passenger_archive(passenger_id, pname, age, archive_date)
VALUES(v_pid, v_name, v_age, CURDATE());
    -> END LOOP;
    -> CLOSE pass_cursor;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE PROCEDURE SafePassengerDelete(IN p_pid INT)
    -> BEGIN
    -> DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    -> BEGIN
    ->     INSERT INTO booking_report(flight_id, passenger_details) VALUES(-1,
CONCAT('Failed to delete passenger ID: ', p_pid, ' due to active bookings constraint'));
    -> END;
    -> DELETE FROM passenger WHERE passenger_id = p_pid;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
```

---

4. Execution and Results: Manifest and Archiving

Executing the procedures to verify cursor behavior and reporting.

```
mysql> CALL GenerateFlightManifest(101);
Query OK, 0 rows affected (0.01 sec)

mysql> CALL GenerateFlightManifest(104);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM booking_report;
+-----------+-----------+-------------------------------------+
| report_id | flight_id | passenger_details                   |
+-----------+-----------+-------------------------------------+
|         1 |       101 | Passenger: Rahul Sharma, Seat: 12A  |
|         2 |       104 | Passenger: Sneha Kulkarni, Seat: 8D |
+-----------+-----------+-------------------------------------+
2 rows in set (0.00 sec)

mysql> CALL ArchiveSeniorPassengers();
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM passenger_archive;
+------------+--------------+--------------+------+--------------+
| archive_id | passenger_id | pname        | age  | archive_date |
+------------+--------------+--------------+------+--------------+
|          1 |            3 | Amit Patil   |   32 | 2026-04-14   |
|          2 |            5 | Vikram Singh |   40 | 2026-04-14   |
```

```
+------------+--------------+--------------+------+--------------+
2 rows in set (0.00 sec)
```

---

5. Execution and Results: Exception Handling

Testing the *SafePassengerDelete* procedure to trigger the exception handler when a passenger has an existing booking.

```
mysql> CALL SafePassengerDelete(1);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM booking_report WHERE flight_id = -1;
+-----------+-----------+-----------------------------------------------------------------+
| report_id | flight_id | passenger_details                                               |
+-----------+-----------+-----------------------------------------------------------------+
|         3 |        -1 | Failed to delete passenger ID: 1 due to active bookings constraint |
+-----------+-----------+-----------------------------------------------------------------+
1 row in set (0.00 sec)
```

Case Study: Civic Voice Interface System

SQL Queries:

1. Audit and Reporting Table Setup

These tables are created to maintain a history of status changes and to store generated system reports or error logs.

```
mysql> USE civic_voice;
Database changed

mysql> CREATE TABLE complaint_audit_log(
    ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     complaint_id INT,
    ->     old_status VARCHAR(20),
    ->     new_status VARCHAR(20),
    ->     change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE system_report(
    ->     report_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     reference_id INT,
    ->     report_details VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.02 sec)
```

---

2. Procedure: Generate Complaint Report (Using Cursors)

This procedure utilizes a **CURSOR** to iterate through joined records of users and complaints to generate a formatted text report for a specific status.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GenerateComplaintReport(IN p_status VARCHAR(20))
    -> BEGIN
    ->     DECLARE done INT DEFAULT 0;
    ->     DECLARE v_user_name VARCHAR(50);
    ->     DECLARE v_title VARCHAR(100);
    ->     DECLARE v_details VARCHAR(255);
    ->     DECLARE complaint_cursor CURSOR FOR SELECT u.name, c.title FROM user u
JOIN complaint c ON u.user_id = c.user_id WHERE c.status = p_status;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->     DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         INSERT INTO system_report(reference_id, report_details) VALUES(-1,
'ERROR GENERATING COMPLAINT REPORT');
    ->     END;
    ->     OPEN complaint_cursor;
    ->     report_loop: LOOP
    ->         FETCH complaint_cursor INTO v_user_name, v_title;
    ->         IF done = 1 THEN
    ->             LEAVE report_loop;
    ->         END IF;
    ->         SET v_details = CONCAT('User: ', v_user_name, ', Issue: ', v_title);
    ->         INSERT INTO system_report(reference_id, report_details) VALUES(0,
v_details);
    ->     END LOOP;
    ->     CLOSE complaint_cursor;
    -> END //
Query OK, 0 rows affected (0.01 sec)
```

---

3. Procedure: Archive Senior Citizens and Exception Handling

This section demonstrates the use of a cursor to archive citizen profiles based on age and an EXCEPTION HANDLER (Error 1062) to manage duplicate entry attempts.

```
mysql> CREATE TABLE citizen_archive(
    ->     archive_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     user_id INT UNIQUE,
    ->     full_name VARCHAR(50),
    ->     age INT,
    ->     archive_date DATE
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE PROCEDURE ArchiveSeniorCitizens()
    -> BEGIN
    ->     DECLARE done INT DEFAULT 0;
    ->     DECLARE v_uid INT;
    ->     DECLARE v_name VARCHAR(50);
    ->     DECLARE v_age INT;
    ->     DECLARE citizen_cursor CURSOR FOR SELECT user_id, full_name, age FROM
citizenprofile WHERE age >= 30;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->     DECLARE CONTINUE HANDLER FOR 1062
    ->     BEGIN
    ->         SET @duplicate_error = 1;
    ->     END;
    ->     OPEN citizen_cursor;
    ->     archive_loop: LOOP
    ->         FETCH citizen_cursor INTO v_uid, v_name, v_age;
    ->         IF done = 1 THEN
    ->             LEAVE archive_loop;
    ->         END IF;
    ->         SET @duplicate_error = 0;
    ->         INSERT INTO citizen_archive(user_id, full_name, age, archive_date)
VALUES(v_uid, v_name, v_age, CURDATE());
    ->     END LOOP;
    ->     CLOSE citizen_cursor;
    -> END //
Query OK, 0 rows affected (0.03 sec)
```

4. Execution and Verification

The following logs verify the execution of the cursor logic and the error handling during a restricted delete operation.

```
mysql> CALL ArchiveSeniorCitizens();
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM citizen_archive;
+------------+---------+------------+------+--------------+
| archive_id | user_id | full_name  | age  | archive_date |
+------------+---------+------------+------+--------------+
|          1 |       3 | Amit Patil |   32 | 2026-04-15   |
+------------+---------+------------+------+--------------+
1 row in set (0.00 sec)

mysql> CREATE PROCEDURE SafeUserDelete(IN p_uid INT)
    -> BEGIN
    ->     DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         INSERT INTO system_report(reference_id, report_details) VALUES(-1,
CONCAT('Failed to delete user ID: ', p_uid, ' due to active records constraint'));
    ->     END;
    ->     DELETE FROM user WHERE user_id = p_uid;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CALL SafeUserDelete(1);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM system_report WHERE reference_id = -1;
+-----------+--------------+-------------------------------------------------------------+
| report_id | reference_id | report_details                                              |
+-----------+--------------+-------------------------------------------------------------+
|         1 |           -1 | Failed to delete user ID: 1 due to active records constraint |
+-----------+--------------+-------------------------------------------------------------+
1 row in set (0.00 sec)
```

Conclusion:

This assignment successfully demonstrated the use of **Cursors** and **Exception Handling** in PL/SQL. While standard SQL queries operate on sets of data, the cursors implemented in procedures like GenerateFlightManifest and ArchiveSeniorCitizens allowed for granular, row-by-row manipulation necessary for report generation and data migration. Furthermore, the integration of exception handlers proved vital for building a resilient system, as seen in the SafeUserDelete procedure, which gracefully manages relational integrity errors. These tools together provide the control and reliability required for enterprise-level database management in both the **Airline** and **Civic Voice** systems.